

A ONE-HOUR TECHNICAL SESSION FOR RF & RAN ENGINEERS

Deep Reinforcement Learning for RAN Planning & Optimization

How DQN and PPO learn to tune coverage, capacity, mobility, interference, and energy the way an experienced RF engineer would — explained without the math.

60 MINUTES · 21 TOPICS · ZERO EQUATIONS REQUIRED

What We'll Cover in 60 Minutes

00–10 min

Why RAN optimization needs AI now, and what RL actually is

10–25 min

DQN and PPO explained simply, with RAN use cases for each

25–45 min

Deep dives: coverage & capacity, mobility, energy, interference, planning

45–55 min

How training really works, risks, guardrails, and a decision guide

55–60 min

Takeaways and open discussion

Why AI Is Entering RAN Now

- ▶ **Complexity has outgrown manual tuning.** Multi-band, multi-vendor, massive MIMO, and dense small cells create far more tunable parameters than a team can hand-tune.
- ▶ **Rule-based SON hits a ceiling.** Static thresholds handle known patterns, but not the combinatorial interactions across thousands of cells.
- ▶ **5G/6G add new control dimensions.** Beamforming, energy targets, network slicing, and sensing all need coordinated, not isolated, tuning.

SCALE OF THE PROBLEM

1000s

of cells per market, each with dozens of tunable parameters

24/7

traffic patterns that shift hourly, daily, and seasonally

Multi-cell

tradeoffs a single-cell fix can't see (coverage vs. interference)

The Limits of Traditional RAN Optimization

- ▶ **Reactive, not consequential.** Rule-based SON and legacy ML respond to known patterns — they don't learn the downstream effect of an action.
- ▶ **Blind to delayed tradeoffs.** Traditional tools optimize what they can see right now, not what happens two rings of cells later.

A FAMILIAR EXAMPLE

You boost power to fix a coverage hole. SINR improves locally — problem solved, according to the rule. But two rings of cells out, interference quietly rises, degrading throughput for neighbors the rule never looked at.

What Is Reinforcement Learning? (Plain-Language Loop)

Agent	The optimization controller making decisions
Environment	The live or simulated RAN — cells, UEs, traffic
State	What it observes: SINR, PRB load, throughput, latency, handovers
Action	A parameter it changes: tilt, power, HO offset, sched. weight
Reward	A score for network health: throughput ↑, drops ↓, energy ↓

The loop: observe → act → get scored → learn → repeat — thousands of times in simulation before ever touching a live network.

Simple RL vs. Deep RL

Simple (Tabular) RL

- ▶ Keeps a lookup table: one best action per state
- ▶ Works only for small, discrete problems
- ▶ Examples: Q-learning, SARSA, Monte Carlo
- ▶ Breaks down once state space gets large

Deep RL

- ▶ Replaces the table with a neural network
- ▶ Generalizes across states it hasn't exactly seen
- ▶ Examples: DQN, PPO, and others
- ▶ Needed the moment state is large or continuous — which is basically always in real RAN data

Two Families of DRL Algorithms

DISCRETE CHOICES

DQN

Picks the best option from a fixed menu of actions

CONTINUOUS CONTROL

PPO

Outputs a precise value and updates behavior gradually and safely

Both learn from trial and error in simulation — they differ in the shape of the action they produce.

DQN Explained Simply

DQN (Deep Q-Network) learns to answer one question: “Given this network state, which of my available discrete actions has historically led to the best long-term outcome?”

BEST SUITED TO

- ▶ Categorical, discrete decisions — not fine continuous values
- ▶ A fixed, enumerable menu of choices at each step
- ▶ Situations where 'which one' matters more than 'how much'

DQN Use Cases in RAN Planning & Optimization

Cell sleep mode

Turn a carrier or cell ON/OFF for energy saving in low traffic

Beam index selection

Choose one of a fixed set of beamforming codebook entries

Discrete power steps

Pick a power level from a preset list (e.g., 5 fixed steps)

Admission control

Admit, queue, or reject a new session

Handover target

Select which neighboring cell to hand a UE to

Scheduling tier

Choose a discrete MCS / priority tier

PPO Explained Simply

PPO (Proximal Policy Optimization) outputs a continuous number directly — the exact tilt, exact power in dBm, exact resource split — rather than choosing from a menu.

WHY “PROXIMAL” MATTERS

- ▶ It constrains how much the policy can change in one update
- ▶ Stays close to the last known-good behavior — no erratic swings
- ▶ Critical in live networks, where a bad update has real consequences

PPO Use Cases in RAN Planning & Optimization

Antenna tilt (CCO)

Continuous tilt optimization for coverage & capacity

Transmit power control

Fine-grained per-cell / per-beam power adjustment

Massive MIMO beams

Beam weight and beam-steering optimization

Energy scaling

Continuous power/PRB scaling with load, not hard on/off

Mobility (MRO)

Tuning HO margins, time-to-trigger, hysteresis

Load balancing

Weights across carriers, bands, or RATs (5G/Wi-Fi)

PART THREE

Five Use Cases, Deep Dive

Coverage · Mobility · Energy · Interference · Planning

Coverage & Capacity Optimization (CCO)

TYPICALLY PPO

The agent learns to balance coverage holes against overshoot interference across a whole cluster of cells — something single-cell tilt optimization can't do, because it never sees the multi-cell coupling that continuous tilt and power changes create.

Mobility Robustness Optimization (MRO)

TYPICALLY PPO

Reduces ping-pong handovers, too-early/too-late handovers, and radio link failures by learning handover parameters per cell-pair from real mobility patterns — instead of one fixed threshold applied everywhere.

Energy Savings

DQN + PPO

Balances energy consumption against QoS by learning when to sleep carriers/cells (DQN) or continuously scale massive MIMO power (PPO) — adapting to each site's actual daily and weekly traffic rhythm rather than a fixed time-based schedule.

Interference Management

TYPICALLY PPO

Learns per-cell power and PRB allocation patterns that reduce inter-cell interference in dense deployments — especially relevant for small-cell clusters and C-band rollouts where classical ICIC struggles to coordinate across many neighbors.

Network Planning Assistance

SIMULATION-DRIVEN

DRL agents trained inside digital twins/simulators rapidly evaluate ‘what if’ site placement, tilt, and power configurations before rollout — compressing planning cycles that used to require manual, iterative drive-testing.

6G: What Changes, What Doesn't

- ▶ **Extreme massive MIMO / sub-THz** Beam management at a scale manual tuning can't reach
- ▶ **Integrated Sensing & Communication (ISAC)** Joint control of sensing and communication parameters
- ▶ **Reconfigurable Intelligent Surfaces (RIS)** Phase-shift optimization across surface elements

All three are naturally continuous control problems — a strong fit for PPO-family methods.

How Training Actually Works

1. Simulate

Train offline in a simulator or digital twin using historical or synthetic traffic — never directly on the live network



2. Validate

Extensive testing in simulation before any real-world exposure



3. Shadow mode

Deployed live but only observing and recommending — not acting



4. Canary rollout

Small cluster deployment, engineers monitoring with rollback ready

Human engineers stay in the loop at every stage, with authority to roll back at any point.

Risks and Practical Guardrails for RF Engineers

- ▶ **Reward shaping pitfalls** A poorly designed reward can optimize the wrong thing — e.g., maximizing throughput while ignoring energy or fairness
- ▶ **Hard safety bounds** Guardrails so the agent can't take extreme actions, like power hard limits
- ▶ **Explainability** Engineers need visibility into why an action was taken, not a black box
- ▶ **Fallback to rules** Revert to rule-based control if the agent behaves unexpectedly
- ▶ **Drift monitoring** Watch for performance decay as traffic patterns change over time

DQN vs. PPO — Quick Reference

	DQN	PPO
Control variable	Discrete / categorical	Continuous / fine-grained
Example	Which cell, which beam index, on/off	Exact power, exact tilt, exact ratio
Update style	Selects from a fixed menu	Smooth, bounded, gradual updates
Best fit for	Simpler sandbox problems first	Production-adjacent, safety-critical rollout

Key Takeaways

- ▶ **DRL is disciplined trial-and-error, not magic.** It outperforms static rules in complex, multi-cell, multi-objective RAN environments.
- ▶ DQN suits discrete decisions; PPO suits continuous, fine-grained, safer control.
- ▶ Success depends more on reward design and staged rollout than on algorithm choice alone.
- ▶ **Start bounded.** Pick one well-instrumented use case — energy savings or tilt optimization — before attempting network-wide control.

OPEN DISCUSSION

Questions & Discussion

Which of your current manual tuning tasks looks most like a DQN problem — and which looks more like a PPO problem?